

Acceleration, FISTA, and Projection

First-Order Methods for Smooth, Non-Smooth, and Constrained Problems

Dr. Robert Hildebrand

Grado Department of Industrial & Systems Engineering
Virginia Tech

Where we are

Last lecture. We built primal-dual interior-point methods. They:

- Solve LP, QP, SOCP, SDP, NLP — almost any convex problem and many nonconvex ones.
- Take **very few iterations** ($\sim 20\text{--}100$).
- Each iteration solves a sparse symmetric-indefinite KKT system: $O(n^3)$ in the dense case, much less when sparse.

What's the catch? The per-iteration cost is the linear solve. For

- LASSO with $p \sim 10^4$ features — IPOPT is fine.
- Image denoising with $n \sim 256^2 = 6.5 \times 10^4$ pixels — IPOPT struggles.
- Training a neural net with $n \sim 10^9$ parameters — IPOPT is hopeless.

Today. *First-order methods.* Per-iteration cost is one gradient evaluation. The trade-off: more iterations needed, but each is cheap.

Today's plot. Three rates that drive practice: $O(1/k)$ (gradient descent), $O(1/k^2)$ (Nesterov), and linear $(1 - c)^k$ (strong convexity). Choosing the right one is the bulk of the algorithm-selection job.

Today's roadmap

1. **The first-order regime** — when and why first-order methods take over from IPMs.
2. **Gradient descent rates** — $O(1/k)$ convex, linear strong-convex. The descent lemma proof.
3. **Heavy-ball / Polyak momentum** — the physical-intuition fix for the zigzag.
4. **Nesterov acceleration** — $O(1/k^2)$. Why it's optimal and what the momentum step really does. When NOT to reach for it.
5. **Composite problems and the proximal operator** — $\min f(x) + g(x)$ for f smooth, g non-smooth (e.g. $\|x\|_1$). Projection as a special case.
6. **ISTA & FISTA** — proximal gradient, then accelerate it.
7. **Parameter-free acceleration** — backtracking, Nesterov's universal method, Lan's adaptive APG.
8. **Coding break.** FISTA from scratch on LASSO; compare with CVXPY.
9. **Three regimes, three winners** — quadratic / logistic / LASSO benchmarks.
10. **Frank–Wolfe preview** — when projection is hard but the linear oracle is cheap.
11. **Wrap-up and Thursday preview.**

Roadmap

The First-Order Regime

Gradient Descent: Rates

Heavy-Ball / Polyak Momentum

Nesterov Acceleration

Composite Problems and the Proximal Operator

ISTA and FISTA

Parameter-Free Acceleration

Coding Break: FISTA from scratch

Three Regimes, Three Winners

Frank–Wolfe (Preview)

Wrap-up

When first-order methods take over

A rough rule-of-thumb chart of solver regimes by problem size:

Variables n	Constraints / cones	Method of choice	Per-iter cost
$\leq 10^3$	general convex, mixed cones	IPOPT / CVXPY-CLARABEL	$O(n^3)$
10^3 – 10^5	sparse convex	IPOPT, MOSEK	$O(\text{nnz}^{1.5})$
10^5 – 10^7	smooth convex	FISTA / L-BFGS	$O(n)$
10^7 – 10^9	smooth + non-smooth (sparsity, TV)	ISTA, FISTA, ADMM	$O(n)$
$\geq 10^9$	finite-sum / streaming	SGD, Adam, SVRG	$O(b)$ batch

What changes as we go down the table.

- *Per-iteration linear-algebra cost* grows from $O(n^3)$ (dense Newton solve) to $O(n)$ (one gradient evaluation) to $O(b)$ (mini-batch).
- *Iteration count* grows from ~ 50 (IPM) to ~ 100 – 10^4 (first-order on smooth) to $\sim 10^5$ – 10^7 (SGD on noisy data).
- *Memory* drops from cubic-storage factorizations to vector-only.

The unifying theme. As problems get bigger, we trade per-iteration accuracy for cheaper iterations. First-order methods are the workhorse of large-scale optimization: every iteration is one gradient.

The setting we'll work in

We focus on **convex composite** problems:

$$\min_{\mathbf{x} \in \mathbb{R}^n} F(\mathbf{x}) := f(\mathbf{x}) + g(\mathbf{x}),$$

where:

- f is convex, differentiable, and **L -smooth**: $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\|_2 \leq L\|\mathbf{x} - \mathbf{y}\|_2$.
- g is convex, possibly non-smooth, but “simple” — meaning we can compute its **proximal map**

$$\text{prox}_{\eta g}(\mathbf{v}) := \arg \min_{\mathbf{u}} g(\mathbf{u}) + \frac{1}{2\eta} \|\mathbf{u} - \mathbf{v}\|_2^2$$

in closed form or in $O(n)$ time.

Examples.

- LASSO: $f(\mathbf{x}) = \frac{1}{2} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|^2$, $g(\mathbf{x}) = \lambda \|\mathbf{x}\|_1$. Prox of ℓ_1 = soft thresholding.
- Constrained: $g(\mathbf{x}) = \mathbb{I}_C(\mathbf{x})$ (indicator of convex set C). Prox = projection onto C .
- TV denoising: f is squared loss, g is the TV (gradient ℓ_1) seminorm. Prox via Chambolle's algorithm.

When $g \equiv 0$ we have plain smooth optimization: gradient descent. When g is the indicator of a set, the framework reduces to projected gradient. The composite framework unifies them.

Roadmap

The First-Order Regime

Gradient Descent: Rates

Heavy-Ball / Polyak Momentum

Nesterov Acceleration

Composite Problems and the Proximal Operator

ISTA and FISTA

Parameter-Free Acceleration

Coding Break: FISTA from scratch

Three Regimes, Three Winners

Frank–Wolfe (Preview)

Wrap-up

Gradient descent — the baseline

For $\min f(\mathbf{x})$ with f L -smooth, the gradient-descent step with constant step size η is

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \eta \nabla f(\mathbf{x}^k).$$

Descent lemma (the workhorse). For L -smooth f and any $\mathbf{x}, \mathbf{y}$:

$$f(\mathbf{y}) \leq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{L}{2} \|\mathbf{y} - \mathbf{x}\|^2.$$

Apply with $\mathbf{y} = \mathbf{x} - \eta \nabla f(\mathbf{x})$:

$$f(\mathbf{x}^{k+1}) \leq f(\mathbf{x}^k) - \eta \left(1 - \frac{L\eta}{2}\right) \|\nabla f(\mathbf{x}^k)\|^2.$$

Choosing $\eta = 1/L$: each step decreases f by at least $\frac{1}{2L} \|\nabla f(\mathbf{x}^k)\|^2$.

Convergence rate (convex case). If f is convex with minimum $f^* = f(\mathbf{x}^*)$,

$$\boxed{f(\mathbf{x}^k) - f^* \leq \frac{L \|\mathbf{x}^0 - \mathbf{x}^*\|^2}{2k}} \quad (\text{rate } O(1/k)).$$

To reach accuracy ε : $k \approx LD^2/(2\varepsilon)$ iterations, where $D = \|\mathbf{x}^0 - \mathbf{x}^*\|$.

Strong convexity — linear rate. If f is also μ -strongly convex ($f - \frac{\mu}{2} \|\cdot\|^2$ convex):

$$\|\mathbf{x}^k - \mathbf{x}^*\|^2 \leq \left(1 - \frac{\mu}{L}\right)^k \|\mathbf{x}^0 - \mathbf{x}^*\|^2.$$

Linear (geometric) rate, with contraction $1 - 1/\kappa$ where $\kappa = L/\mu$ is the **condition number**. To reach ε : $k \approx \kappa \log(1/\varepsilon)$.

Two regimes, two rates. Convex but not strongly convex: $O(1/k)$ — you need $1/\varepsilon$ iterations for accuracy ε . Strongly convex: linear — $\kappa \log(1/\varepsilon)$ iterations.

What goes wrong — the narrow-valley picture

Gradient descent on a narrow ellipsoidal quadratic

$$f(\mathbf{x}) = \frac{1}{2}(x_1^2 + \kappa x_2^2), \quad \kappa \gg 1$$

zig-zags: the gradient points mostly across the valley, not along it.

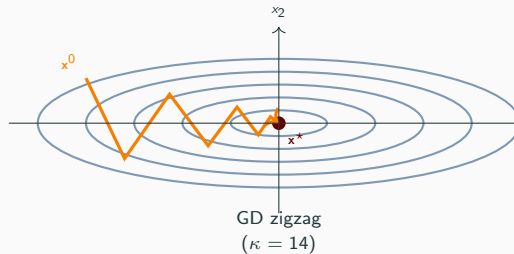
Step size dilemma.

- $\eta = 1/L = 1/\kappa$ is safe but slow along the long axis.
- Larger η overshoots in the steep direction.

Result. Linear rate $1 - 1/\kappa$ — progress is bottlenecked by the worst-conditioned direction.

For ill-conditioned f ($\kappa = 1000$): each step makes 0.1% progress. Need $\sim \kappa$ iterations to halve the error. Hence the appeal of:

- Newton ($O(\sqrt{\kappa})$ effective via affine-invariance — but per-iter cost too high).



Roadmap

The First-Order Regime

Gradient Descent: Rates

Heavy-Ball / Polyak Momentum

Nesterov Acceleration

Composite Problems and the Proximal Operator

ISTA and FISTA

Parameter-Free Acceleration

Coding Break: FISTA from scratch

Three Regimes, Three Winners

Frank–Wolfe (Preview)

Wrap-up

Heavy-ball: a physical fix for the zigzag

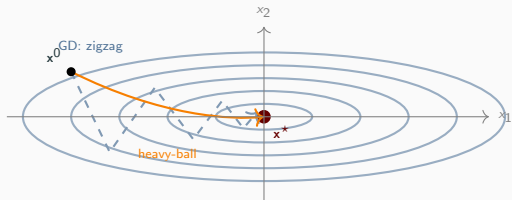
Polyak (1964). The first momentum method. Add an inertia term to plain GD:

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \alpha \nabla f(\mathbf{x}^k) + \beta (\mathbf{x}^k - \mathbf{x}^{k-1}).$$

$\beta \in [0, 1)$: how much of the previous step you carry over. $\alpha > 0$: gradient step size as before.

Physical analogy. Imagine a heavy ball rolling down the loss surface. Gradient = force; mass gives the ball inertia.

- In a narrow valley, plain GD's gradient points across the valley, causing zigzag.
- Heavy-ball's previous-step term keeps you moving *along* the valley even when the gradient pulls sideways. Inertia averages out the cross-valley component.



Heavy-ball averages the cross-valley oscillation away by carrying the previous step's momentum into the current step.

Optimal tuning on a strongly-convex quadratic

What heavy-ball does and doesn't buy

Strongly-convex quadratics. Heavy-ball with α^*, β^* achieves the optimal $\sqrt{\kappa}$ rate, matching Nesterov. *No advantage of Nesterov on this class.*

General smooth convex (not strongly convex). Heavy-ball does **not** achieve $O(1/k^2)$. The best-tuned heavy-ball is stuck at $O(1/k)$ — same as plain GD. *This is where Nesterov wins.*

Why the difference. Heavy-ball evaluates the gradient at $\mathbf{x}^k$. Nesterov evaluates it at the *look-ahead* point $\mathbf{y}^k = \mathbf{x}^k + \beta_k(\mathbf{x}^k - \mathbf{x}^{k-1})$. The lookahead is what gives Nesterov the extra factor of $\sqrt{k}$ on general convex problems — by the time you compute the gradient, the iterate has already moved with the momentum, so the gradient “sees” the future direction.

Lessons heavy-ball teaches.

- Momentum is essentially a way to **average out cross-valley noise** in the gradient direction.
- For *quadratics*, the right amount of momentum is enough: heavy-ball is optimal.
- For *general convex problems*, you need momentum **plus** a forward-looking gradient evaluation: that's Nesterov.
- Modern practice on smooth nonconvex problems (deep learning): the look-ahead doesn't help much because of stochastic noise and nonconvexity. Many SGD implementations **use heavy-ball, not Nesterov**, with constant $\beta = 0.9$.

In summary. Heavy-ball: simpler, optimal on quadratics, the standard “momentum” in deep learning. Nesterov: cleverer, optimal across the entire smooth convex class. We'll see Nesterov's modification next, where the lookahead trick is the only change.

Roadmap

The First-Order Regime

Gradient Descent: Rates

Heavy-Ball / Polyak Momentum

Nesterov Acceleration

Composite Problems and the Proximal Operator

ISTA and FISTA

Parameter-Free Acceleration

Coding Break: FISTA from scratch

Three Regimes, Three Winners

Frank–Wolfe (Preview)

Wrap-up

Nesterov 1983 — one extra evaluation, $O(1/k^2)$ rate

One might expect gradient descent to be *first-order optimal* on smooth convex problems. It isn't.

The lower bound is $O(1/k^2)$ on the number of gradient evaluations, not $O(1/k)$ (Nemirovski-Yudin). Gradient descent leaves a factor of k on the table.

Nesterov's method (1983). Maintain two sequences. With L -smooth f and step $\eta = 1/L$:

$$\begin{aligned} \mathbf{y}^k &= \mathbf{x}^k + \frac{t_{k-1} - 1}{t_k}(\mathbf{x}^k - \mathbf{x}^{k-1}) \quad (\text{extrapolation / momentum}) \\ \mathbf{x}^{k+1} &= \mathbf{y}^k - \frac{1}{L}\nabla f(\mathbf{y}^k) \quad (\text{gradient step from } \mathbf{y}^k, \text{ not } \mathbf{x}^k) \end{aligned}$$

where the momentum coefficient comes from the recursion

$$t_0 = 1, \quad t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2} \approx \frac{k+2}{2}.$$

Theorem (Nesterov 1983). For convex L -smooth f with minimizer $\mathbf{x}^*$,

$$f(\mathbf{x}^k) - f^* \leq \frac{2L\|\mathbf{x}^0 - \mathbf{x}^*\|^2}{(k+1)^2} \quad (\text{rate } O(1/k^2)).$$

To reach accuracy ε : $k \approx \sqrt{LD^2/\varepsilon}$ instead of LD^2/ε .

Why $O(1/k^2)$ matters. For $\varepsilon = 10^{-6}$, $LD^2 = 1$: GD needs 5×10^5 iterations; Nesterov needs $\sim 10^3$. Two-orders-of-magnitude speedup at *no* extra per-iteration cost — still one gradient.

The Nesterov step, point by point

At iteration k we have two stored iterates: $\mathbf{x}^{k-1}$ and $\mathbf{x}^k$. Three things happen.

1. **Extrapolate:** look ahead from $\mathbf{x}^k$ in the direction of the previous step.

$$\mathbf{y}^k = \mathbf{x}^k + \beta_k(\mathbf{x}^k - \mathbf{x}^{k-1}), \quad \beta_k = \frac{t_{k-1}-1}{t_k}.$$

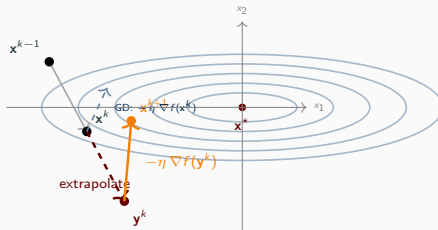
The extrapolated point $\mathbf{y}^k$ is **not** a candidate iterate — it's just where we'll evaluate the gradient.

2. **Evaluate the gradient at $\mathbf{y}^k$, not at $\mathbf{x}^k$:** compute $\nabla f(\mathbf{y}^k)$. This is the only gradient computation per iteration. *Plain GD would have evaluated at $\mathbf{x}^k$.*

3. **Take a gradient step from $\mathbf{y}^k$:**

$$\mathbf{x}^{k+1} = \mathbf{y}^k - \eta \nabla f(\mathbf{y}^k).$$

Why this is faster. The gradient at $\mathbf{y}^k$ “incorporates” some of the history because $\mathbf{y}^k$ already contains the previous step direction. On the narrow-valley problem, the gradient at $\mathbf{x}^k$ points across the valley (causing zigzag); the gradient at $\mathbf{y}^k$ has been “rotated” toward the long axis by the look-ahead.



- **Black dots:** stored iterates $\mathbf{x}^{k-1}$, $\mathbf{x}^k$. Gray arrow: previous step.
- **Maroon:** extrapolated look-ahead $\mathbf{y}^k$ (dashed arrow). **No gradient is evaluated at $\mathbf{x}^k$.**
- **Orange:** gradient at $\mathbf{y}^k$ and the new iterate $\mathbf{x}^{k+1}$.
- **Blue dashed:** what plain GD would have done — gradient at $\mathbf{x}^k$, perpendicular to the contour, going across the valley.

The key picture. GD's gradient at $\mathbf{x}^k$ points across the narrow valley (perpendicular to the long contour). Nesterov's gradient at $\mathbf{y}^k$ — a point that's already moved further in the previous-step direction — is at a flatter contour, so its gradient has *less* cross-valley component. Adding the look-ahead is what cancels the zigzag.

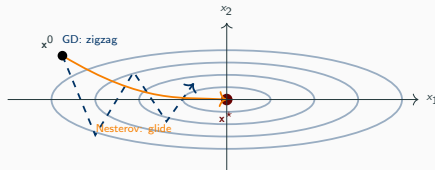
GD vs. Nesterov: full trajectories

The previous slide showed one iteration in detail.
Here is what *many* iterations look like, started from the same $\mathbf{x}^0$ on the same narrow-valley problem.

What to notice.

- GD bounces across the long axis of the valley. Every step undoes part of the previous one.
- Nesterov spends a few iterations turning into the long axis, then glides down it. The momentum doesn't reset between steps.
- The number of steps to reach $\mathbf{x}^*$ is dramatically different — this is the geometric content of $O(1/k)$ vs. $O(1/k^2)$.

Same step size $\eta = 1/L$. Same starting point. Only difference: where the gradient is evaluated.



Connection to the rates. The zigzag pattern is exactly why GD's worst-case rate has κ in it (linear) or L in it (sublinear). Nesterov's gliding path is what the $\sqrt{\kappa} / \sqrt{L/\varepsilon}$ scaling buys you geometrically.

The big picture: rates at a glance

Iterations to reach error ε on smooth convex f :

Method	Convex	μ -strongly convex
Gradient descent ($\eta = 1/L$)	$O(LD^2/\varepsilon)$	$O(\kappa \log(1/\varepsilon))$
Heavy-ball (tuned)	$O(\sqrt{LD^2/\varepsilon})^*$	$O(\sqrt{\kappa} \log(1/\varepsilon))^*$
Nesterov / FISTA	$O(\sqrt{LD^2/\varepsilon})$	$O(\sqrt{\kappa} \log(1/\varepsilon))$
Newton (per-iter cost $O(n^3)$)	$O(\log \log(1/\varepsilon))^{**}$	$O(\log \log(1/\varepsilon))^{**}$

* Heavy-ball achieves $\sqrt{\kappa}$ on quadratics, but *not* on general convex. ** In the quadratic-convergence region. † The strongly-convex column requires the constant-momentum variant of Nesterov ($\beta = (1 - \sqrt{\mu/L})/(1 + \sqrt{\mu/L})$), not the t_k recursion shown earlier.

Take-away

Per-iteration cost is essentially the same for all first-order methods (one gradient). The huge differences in iteration count come from *exploiting structure*: smoothness, strong convexity, and the right kind of momentum.

Lower bound. Nemirovski-Yudin: any first-order method needs $\Omega(1/k^2)$ on smooth convex and $\Omega(\sqrt{\kappa})$ on strongly convex. Nesterov *matches* both — so it's optimal.

When NOT to use Nesterov — Part 1: gradient descent often wins

Nesterov is *worst-case optimal* for first-order smooth convex — but “optimal” is a worst-case statement over a function class. There are real situations where plain gradient descent beats it.

Choose plain gradient descent when:

- *Gradients are noisy.* Stochastic, sub-sampled, or computed approximately. Nesterov's look-ahead amplifies noise; the $O(1/k^2)$ rate degrades to roughly $O(1/k)$ and iterates become unstable. SGD-without-momentum is safer (more on this Thursday).
- *You need monotone progress.* GD with $\eta = 1/L$ guarantees $f(\mathbf{x}^{k+1}) \leq f(\mathbf{x}^k)$. FISTA can have the objective *increase* on individual iterations — still converges, but unsettling and harder to debug. Restart strategies fix this but add tuning.
- *You restart frequently.* Nesterov builds up momentum over iterations. If you warm-start from a near-optimum or re-solve a perturbed problem, the momentum is stale and counter-productive. GD has no state to carry across restarts.
- *The look-ahead point $\mathbf{y}^k$ lands in bad regions.* Outside a chart, beyond a barrier, in invalid parameter space. $\mathbf{y}^k$ can violate constraints that $\mathbf{x}^k$ satisfies.

When NOT to use Nesterov — Part 2: CG and L-BFGS

Choose conjugate gradient (CG) when the problem is quadratic. For $\min \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x} - \mathbf{b}^\top \mathbf{x}$ with $\mathbf{A} \succ 0$ (or equivalently solving $\mathbf{A} \mathbf{x} = \mathbf{b}$):

- *Same per-iteration cost as Nesterov* — one matrix-vector product. Same $\sqrt{\kappa} \log(1/\varepsilon)$ asymptotic rate.
- *Parameter-free.* No L to estimate; the step length and search direction are computed automatically each iteration.
- *Exact in $\leq n$ iterations* in exact arithmetic. Nesterov never finishes.
- *Adapts to spectral structure.* If $\mathbf{A}$ has only k distinct eigenvalues, CG converges in exactly k steps. Nesterov sees only the worst-case $\sqrt{\kappa}$.

For most LP/QP IPM inner solves, CG (or preconditioned CG) is the linear-system solver of choice — not Nesterov.

Choose quasi-Newton (L-BFGS) when f is smooth and you can afford to remember ~ 10 – 20 past gradient differences. L-BFGS builds an implicit local metric that adapts to the problem's curvature; on poorly scaled problems, this implicit preconditioning can beat Nesterov by an order of magnitude. Most off-the-shelf ML training (logistic regression, GLMs, Cox models) uses L-BFGS, not Nesterov.

Where Nesterov wins: large-scale *composite* problems ($f + g$, f smooth and g non-smooth, e.g. $g = \|\mathbf{x}\|_1$) where Newton-type methods and CG don't apply directly. The accelerated version of proximal gradient on these problems — which we'll build in the next two sections under the name FISTA — is the right tool for LASSO, TV denoising, sparse regression, low-rank matrix recovery. Logistic regression on standard data: probably L-BFGS.

Honest summary. Nesterov is the right answer when (a) gradients are exact, (b) the problem is convex and smooth (or composite with cheap prox), (c) the dimension is too big for Newton-type methods, and (d) you're not in a stochastic regime. Outside that pocket, GD / CG / L-BFGS often beat it on the practical metric (wall-clock time to your accuracy target).

Roadmap

The First-Order Regime

Gradient Descent: Rates

Heavy-Ball / Polyak Momentum

Nesterov Acceleration

Composite Problems and the Proximal Operator

ISTA and FISTA

Parameter-Free Acceleration

Coding Break: FISTA from scratch

Three Regimes, Three Winners

Frank–Wolfe (Preview)

Wrap-up

The proximal operator: definition

For a convex function $g : \mathbb{R}^n \rightarrow \mathbb{R} \cup \{+\infty\}$ and $\eta > 0$, the **proximal operator** is

$$\text{prox}_{\eta g}(\mathbf{v}) := \arg \min_{\mathbf{u}} \left\{ g(\mathbf{u}) + \frac{1}{2\eta} \|\mathbf{u} - \mathbf{v}\|_2^2 \right\}.$$

Reading. The point closest to $\mathbf{v}$ that has small g -value, with the trade-off controlled by η . Always uniquely defined for proper closed convex g .

Three foundational examples.

- $g(\mathbf{u}) = 0$. $\text{prox}_{\eta 0}(\mathbf{v}) = \mathbf{v}$. Identity.
- $g(\mathbf{u}) = \mathbb{I}_C(\mathbf{u})$ (**indicator of a closed convex set C**). $\text{prox}_{\eta g}(\mathbf{v}) = \arg \min_{\mathbf{u} \in C} \|\mathbf{u} - \mathbf{v}\|^2 =$ *Euclidean projection* onto C , written $\Pi_C(\mathbf{v})$. Independent of η .
- $g(\mathbf{u}) = \lambda \|\mathbf{u}\|_1$. Componentwise *soft-thresholding*:

$$[\text{prox}_{\eta g}(\mathbf{v})]_i = \text{sign}(v_i) \max(|v_i| - \eta\lambda, 0).$$

ℓ_1 regularization shrinks coordinates toward zero, with anything below $\eta\lambda$ in magnitude crushed to exactly zero — the source of LASSO's sparsity.

Why this object? The prox replaces the “project this gradient step onto where the non-smooth penalty is small” move that arises naturally in optimizing $f + g$ with non-smooth g .

Worked prox: ℓ_1 soft-thresholding (full derivation)

For $g(\mathbf{u}) = \lambda \|\mathbf{u}\|_1$, the prox separates by coordinate. Each scalar problem:

$$\min_u \lambda |u| + \frac{1}{2\eta}(u - v)^2.$$

Subgradient optimality: $0 \in \lambda \partial|u^*| + (u^* - v)/\eta$ where $\partial|0| = [-1, 1]$.

Three cases by the sign and magnitude of v :

- $v > \eta\lambda$: optimum $u^* > 0$, $\partial|u^*| = \{1\}$, so $u^* = v - \eta\lambda$.
- $v < -\eta\lambda$: optimum $u^* < 0$, $\partial|u^*| = \{-1\}$, so $u^* = v + \eta\lambda$.
- $|v| \leq \eta\lambda$: $u^* = 0$ is optimal because $0 \in \lambda \cdot [-1, 1] - v/\eta$.

Combined: $u^* = \text{sign}(v) \max(|v| - \eta\lambda, 0)$. ✓

Why this matters. The soft-threshold map shrinks every coordinate by $\eta\lambda$ toward zero. Coordinates with $|v_i| < \eta\lambda$ get *exactly* zeroed out — this is the source of LASSO's sparsity. Larger $\lambda \Rightarrow$ more aggressive zeroing $\Rightarrow$ sparser solution.

Catalogue. Almost every non-smooth g used in practice has a known closed-form or $O(n \log n)$ prox: ℓ_1 , ℓ_∞ , group- ℓ_1 , nuclear norm, indicator of box / ball / simplex / SOC / PSD cone, total variation (via Chambolle's algorithm). Memorize the first few; look up the rest.

Why ℓ_1 produces sparsity — the geometric picture

The prox of ℓ_1 is soft-thresholding.

$$[\text{prox}_{\eta\lambda\|\cdot\|_1}(\mathbf{v})]_i = \text{sign}(v_i) (|v_i| - \eta\lambda)_+.$$

- Coordinates with $|v_i| \leq \eta\lambda$ get crushed to **exactly zero**.
- Larger coordinates shrink toward zero by a constant amount $\eta\lambda$.

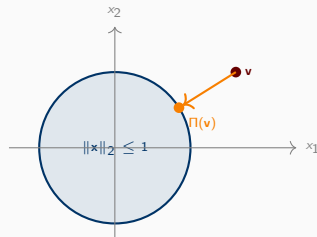
Why ℓ_1 and not ℓ_2 ? The geometry of the unit balls.

- ℓ_2 ball: smooth boundary, no preferred axis. The closest point on the ball to a generic $\mathbf{v}$ is generically dense.
- ℓ_1 ball: **corners on the axes**. The closest point from a generic $\mathbf{v}$ often lands on a corner — a *sparse* vector.

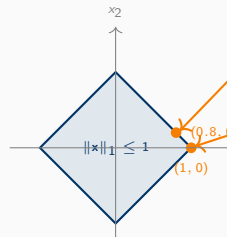
Equivalent formulation. The constrained LASSO

$$\min_{\mathbf{x}} \frac{1}{2} \|\mathbf{Ax} - \mathbf{b}\|^2 \text{ s.t. } \|\mathbf{x}\|_1 \leq r$$

has the same solution path as the regularized one



ℓ_2 : smooth ball
projection is generically *dense*



ℓ_1 : corners on axes
projection often hits a corner $\Rightarrow$ *sparse*

The mechanism. For any $\mathbf{v}$ in the open “corner cone” near a ℓ_1 -ball vertex, the projection lands exactly on that vertex — a sparse point. As you scan $\mathbf{v}$ over $\mathbb{R}^n$, vertex projections occupy a positive-measure region. For the ℓ_2 ball, vertices don't exist, so the projection is generically dense.

This is the geometric origin of LASSO sparsity.

Projection: the most important special case

When g is the indicator of a closed convex set C :

$$g(\mathbf{u}) = \mathbb{I}_C(\mathbf{u}) = \begin{cases} 0, & \mathbf{u} \in C, \\ +\infty, & \mathbf{u} \notin C. \end{cases}$$

the prox simplifies to the **Euclidean projection**:

$$\text{prox}_{\eta g}(\mathbf{v}) = \arg \min_{\mathbf{u} \in C} \frac{1}{2} \|\mathbf{u} - \mathbf{v}\|_2^2 =: \Pi_C(\mathbf{v}).$$

This is the closest point in C to $\mathbf{v}$. The penalty η disappears (any positive η gives the same minimizer): when you must be in C , the only thing left is to be as close to $\mathbf{v}$ as possible.

Projection turns ISTA into projected gradient. For $\min f(\mathbf{x})$ s.t. $\mathbf{x} \in C$:

$$\mathbf{x}^{k+1} = \Pi_C(\mathbf{x}^k - \eta \nabla f(\mathbf{x}^k)).$$

Take a step in $-\nabla f$, then snap back to the feasible set. This is one of the simplest first-order methods for constrained problems.

Why projection is so useful. The projected-gradient operator $T(\mathbf{x}) = \Pi_C(\mathbf{x} - \eta \nabla f(\mathbf{x}))$ inherits “contractive” behavior from Π_C : distances between iterates do not grow. Iterating T converges to a fixed point that's a stationary point of the constrained problem (when η is small enough). Crucially, the same convergence theorems as plain gradient descent — $O(1/k)$ rate, linear under strong convexity — carry over.

Behind every constrained-optimization method (projected gradient, mirror descent, conditional gradient) is some flavor of projection or its generalization. The cost of projection determines whether you can use it.

A catalogue of cheap projections

Set C	Definition	$\Pi_C(\mathbf{v})$	Cost
Nonneg orthant	$\{\mathbf{u} \geq 0\}$	$\max(\mathbf{v}, 0)$ componentwise	$O(n)$
Box	$\{\mathbf{u} : \ell_i \leq u_i \leq u_i^{\max}\}$	clamp each v_i to $[\ell_i, u_i^{\max}]$	$O(n)$
Half-space	$\{\mathbf{u} : \mathbf{a}^\top \mathbf{u} \leq b\}$	$\mathbf{v} - \frac{(\mathbf{a}^\top \mathbf{v} - b)_+}{\ \mathbf{a}\ ^2} \mathbf{a}$	$O(n)$
Hyperplane	$\{\mathbf{u} : \mathbf{a}^\top \mathbf{u} = b\}$	$\mathbf{v} - \frac{\mathbf{a}^\top \mathbf{v} - b}{\ \mathbf{a}\ ^2} \mathbf{a}$	$O(n)$
Affine subspace	$\{\mathbf{u} : \mathbf{A}\mathbf{u} = \mathbf{b}\}$	$\mathbf{v} - \mathbf{A}^\top (\mathbf{A}\mathbf{A}^\top)^{-1} (\mathbf{A}\mathbf{v} - \mathbf{b})$	$O(n) + \text{setup}$
Euclidean ball	$\{\mathbf{u} : \ \mathbf{u}\ _2 \leq r\}$	$\min(1, r/\ \mathbf{v}\ _2) \cdot \mathbf{v}$	$O(n)$
Probability simplex	$\{\mathbf{u} \geq 0, \sum u_i = 1\}$	$(\mathbf{v} - \tau \mathbf{1})_+$ with τ s.t. $\sum = 1$	$O(n \log n)$
ℓ_1 ball	$\{\mathbf{u} : \ \mathbf{u}\ _1 \leq r\}$	soft-threshold with τ s.t. $\sum \cdot = r$	$O(n \log n)$
Second-order cone	$\{(t, \mathbf{u}) : t \geq \ \mathbf{u}\ _2\}$	closed-form three-case formula	$O(n)$
PSD cone	$\{X \succeq 0\}$	$\sum_i \max(\lambda_i, 0) \mathbf{v}_i \mathbf{v}_i^\top$ via eig	$O(n^3)$

Walking through three:

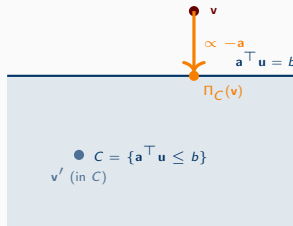
- **Nonneg orthant.** The closest nonneg point to $\mathbf{v}_i$ is $\max(v_i, 0)$ — if $v_i \geq 0$ keep it, if $v_i < 0$ snap to 0. Independently per coordinate.
- **Half-space.** If $\mathbf{v}$ already satisfies the constraint, leave it. Otherwise drop a perpendicular onto the boundary hyperplane: subtract $(\mathbf{a}^\top \mathbf{v} - b)/\|\mathbf{a}\|^2$ times the normal $\mathbf{a}$.
- **Probability simplex.** The Lagrangian KKT conditions for the QP $\min \frac{1}{2} \|\mathbf{u} - \mathbf{v}\|^2$ s.t. $\mathbf{u} \geq 0, \mathbf{1}^\top \mathbf{u} = 1$ give $\mathbf{u}^* = (\mathbf{v} - \tau \mathbf{1})_+$, where τ is the unique scalar making the sum equal 1. Sort $\mathbf{v}$ in decreasing order and pick the right pivot — $O(n \log n)$.

Reading the table. Sets that decompose coordinatewise (orthant, box) project trivially. Sets defined by a single linear constraint (half-space, hyperplane) require one inner product. Sets with a sum constraint (simplex, ℓ_1 ball) take an extra $O(n \log n)$ to find the threshold. Spectral sets (PSD cone, nuclear ball) cost an eigendecomposition or SVD.

Projection: a picture

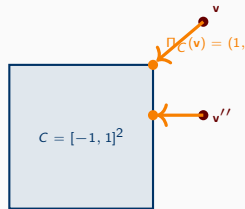
Half-space $C = \{\mathbf{u} : \mathbf{a}^\top \mathbf{u} \leq b\}$.

- If $\mathbf{v}$ is on the right side, $\Pi_C(\mathbf{v}) = \mathbf{v}$ (no movement).
- If $\mathbf{v}$ is on the wrong side, drop a perpendicular onto the boundary hyperplane.
- The displacement is along the normal $\mathbf{a}$ — shortest path to feasibility.



Half-space: drop a perpendicular

Box / orthant. Project onto a Cartesian product = independently project onto each factor. Component-wise clamping. The picture: each coordinate moves perpendicular to its own constraint face.



Box: clamp each coordinate independently

Geometric reading. The variational characterization

$$\langle \mathbf{v} - \Pi_C(\mathbf{v}), \mathbf{y} - \Pi_C(\mathbf{v}) \rangle \leq 0 \quad \forall \mathbf{y} \in C$$

says: the residual vector forms an obtuse angle with every direction into C . There's nothing closer.

Why these are the easy cases. The half-space projection is one inner product.

Box and orthant separate by coordinate — $O(n)$. Sets with a single linear constraint or a coordinatewise structure cover most of the easy column in the catalogue.

Why projection is a “well-behaved” operator

The projection Π_C has three structural properties that make it safe to use as a building block in iterative algorithms.

(1) **Variational characterization.** For closed convex C and any $\mathbf{v} \in \mathbb{R}^n$:

$$\mathbf{x}^* = \Pi_C(\mathbf{v}) \iff \mathbf{x}^* \in C \text{ and } \langle \mathbf{v} - \mathbf{x}^*, \mathbf{y} - \mathbf{x}^* \rangle \leq 0 \quad \forall \mathbf{y} \in C.$$

Geometrically: at $\mathbf{x}^*$, the residual vector $\mathbf{v} - \mathbf{x}^*$ makes an obtuse angle with every direction into C . There's no way to move into C that brings you closer to $\mathbf{v}$.

(2) **Non-expansive.** For all $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$:

$$\|\Pi_C(\mathbf{u}) - \Pi_C(\mathbf{v})\|_2 \leq \|\mathbf{u} - \mathbf{v}\|_2.$$

Projection *never increases* pairwise distances. (*Proof.* Apply (1) twice and take a Cauchy-Schwarz; or just observe that the projection map is the identity on C and contracts orthogonally to C .)

(3) **Idempotent.** $\Pi_C(\Pi_C(\mathbf{v})) = \Pi_C(\mathbf{v})$ — projecting twice is the same as projecting once. Once you're in C , you stay in C .

Why these matter. The projected-gradient operator $T(\mathbf{x}) = \Pi_C(\mathbf{x} - \eta \nabla f(\mathbf{x}))$ is a composition of:

- a gradient step, which is a contraction by factor $1 - \eta\mu$ for μ -strongly-convex f ,
- the projection Π_C , which is non-expansive.

Composition is still contractive, so T has a unique fixed point, and Picard iteration $\mathbf{x}^{k+1} = T(\mathbf{x}^k)$ converges to it. *This is why projected gradient just works* — not because we proved a clever rate, but because the projection “fits inside” the gradient-descent contraction argument without breaking it.

Generalization. The same is true for the proximal operator: $\text{prox}_{\eta g}$ is non-expansive (in fact, *firmly* non-expansive) for any closed convex g . ISTA, FISTA, ADMM, and the splitting methods are built on this single fact.

Prox calculus and the Moreau decomposition

Useful calculus rules (Vandenberghe ECE236C, Lec. 6).

- *Separable sum:* $f(\mathbf{x}, \mathbf{y}) = g(\mathbf{x}) + h(\mathbf{y}) \Rightarrow \text{prox}_f(\mathbf{x}, \mathbf{y}) = (\text{prox}_g(\mathbf{x}), \text{prox}_h(\mathbf{y}))$.
- *Affine substitution:* $f(\mathbf{x}) = g(\mathbf{a}\mathbf{x} + \mathbf{b})$ gives $\text{prox}_f(\mathbf{x}) = \frac{1}{a}(\text{prox}_{a^2g}(\mathbf{a}\mathbf{x} + \mathbf{b}) - \mathbf{b})$.
- *Linear shift:* $f(\mathbf{x}) = g(\mathbf{x}) + \mathbf{a}^\top \mathbf{x} \Rightarrow \text{prox}_f(\mathbf{x}) = \text{prox}_g(\mathbf{x} - \mathbf{a})$.
- *Quadratic addition:* $f(\mathbf{x}) = g(\mathbf{x}) + \frac{\mu}{2}\|\mathbf{x} - \mathbf{a}\|^2 \Rightarrow \text{prox}_f(\mathbf{x}) = \text{prox}_{\theta g}(\theta\mathbf{x} + (1 - \theta)\mathbf{a})$ with $\theta = 1/(1 + \mu)$.

Logarithmic barrier prox. $f(\mathbf{x}) = -\sum_i \log x_i$ gives $\text{prox}_{tf}(\mathbf{x})_i = (x_i + \sqrt{x_i^2 + 4t})/2$.

Moreau decomposition (the spectacular identity):

$$\mathbf{x} = \text{prox}_f(\mathbf{x}) + \text{prox}_{f^*}(\mathbf{x}) \quad \forall \mathbf{x}, f \text{ closed convex.}$$

where $f^*(\mathbf{y}) = \sup_{\mathbf{x}}(\mathbf{y}^\top \mathbf{x} - f(\mathbf{x}))$ is the convex conjugate.

Why striking. Generalizes the subspace-decomposition $\mathbf{x} = P_L(\mathbf{x}) + P_{L^\perp}(\mathbf{x})$: with $f = \mathbb{I}_L$ we have $f^* = \mathbb{I}_{L^\perp}$ and the prox formulas reduce to projections.

Practical use. Free formula for prox_{f^*} once you have prox_f . Underpins Douglas-Rachford and ADMM (Thursday).

Roadmap

The First-Order Regime

Gradient Descent: Rates

Heavy-Ball / Polyak Momentum

Nesterov Acceleration

Composite Problems and the Proximal Operator

ISTA and FISTA

Parameter-Free Acceleration

Coding Break: FISTA from scratch

Three Regimes, Three Winners

Frank–Wolfe (Preview)

Wrap-up

ISTA: proximal gradient descent

Goal. Minimize $F(\mathbf{x}) = f(\mathbf{x}) + g(\mathbf{x})$ with f smooth, g “simple”.

Proximal gradient (ISTA — Iterative Soft-Thresholding Algorithm). For step size $\eta \leq 1/L$:

$$\mathbf{x}^{k+1} = \text{prox}_{\eta g}(\mathbf{x}^k - \eta \nabla f(\mathbf{x}^k)).$$

Reading. Take a gradient step on f , then “snap” the result to a low- g -value point via the prox. When $g \equiv 0$, this is plain gradient descent. When $g = \mathbb{I}_C$, it's projected gradient: $\mathbf{x}^{k+1} = \Pi_C(\mathbf{x}^k - \eta \nabla f(\mathbf{x}^k))$.

Convergence. Beck–Teboulle (2009): for convex L -smooth f and convex g , with $\eta = 1/L$:

$$F(\mathbf{x}^k) - F^* \leq \frac{L \|\mathbf{x}^0 - \mathbf{x}^*\|^2}{2k} \quad (O(1/k) \text{ rate}).$$

Same as plain gradient descent in the smooth case — the prox doesn't slow you down.

LASSO via ISTA. For $f = \frac{1}{2} \|\mathbf{Ax} - \mathbf{b}\|^2$, $g = \lambda \|\mathbf{x}\|_1$:

$$\mathbf{x}^{k+1} = \text{soft}_{\eta \lambda}(\mathbf{x}^k - \eta \mathbf{A}^\top (\mathbf{Ax}^k - \mathbf{b})),$$

where $\text{soft}_\tau(\mathbf{u})_i = \text{sign}(u_i) \max(|u_i| - \tau, 0)$. One matrix-vector multiply + one shrinkage per iteration. Sparsity emerges automatically from the soft-threshold: small coordinates get zeroed out.

FISTA: ISTA + Nesterov momentum

The recipe. Apply Nesterov-style extrapolation to the ISTA iteration. Beck & Teboulle (2009):

$$\begin{aligned}\mathbf{x}^{k+1} &= \text{prox}_{\eta g}(\mathbf{y}^k - \eta \nabla f(\mathbf{y}^k)) \\ t_{k+1} &= \frac{1 + \sqrt{1 + 4t_k^2}}{2} \\ \mathbf{y}^{k+1} &= \mathbf{x}^{k+1} + \frac{t_k - 1}{t_{k+1}}(\mathbf{x}^{k+1} - \mathbf{x}^k)\end{aligned}$$

with $t_0 = 1$, $\mathbf{y}^0 = \mathbf{x}^0$.

Theorem. For convex L -smooth f and convex g , with $\eta = 1/L$:

$$F(\mathbf{x}^k) - F^* \leq \frac{2L\|\mathbf{x}^0 - \mathbf{x}^*\|^2}{(k+1)^2} \quad O(1/k^2) \text{ rate.}$$

No extra cost per iteration over ISTA. Same one gradient, same one prox.

For LASSO. On $n = 10^4$ features, ISTA needs $\sim 5,000$ iterations to reach 10^{-6} ; FISTA needs ~ 200 . The factor-of-25 speedup is exactly the $O(1/k) \rightarrow O(1/k^2)$ improvement.

Practical caveat. FISTA's monotonicity is not guaranteed — the objective can occasionally increase along the iteration. Restart strategies (Beck & Teboulle 2009, O'Donoghue & Candès 2015) recover monotone descent and often improve constants in practice.

The t_k recursion: what the numbers actually look like

Recursion. $t_0 = 1$,

$$t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}.$$

The momentum coefficient at step k is

$$\beta_k = (t_{k-1} - 1)/t_k.$$

What grows, and how fast.

- t_k linearly in k : $t_k \approx (k + 2)/2$.
- β_k approaches 1: $\beta_k \rightarrow (k - 1)/(k + 1) \rightarrow 1$ as $k \rightarrow \infty$.
- In the convergence proof, only $t_k^2 \approx (k + 2)^2/4$ matters, and that's what gives the $O(1/k^2)$ rate.

Reading the table. At iteration 5 we already have $\beta \approx 0.6$ (substantial momentum). By iteration 15 we're at $\beta \approx 0.83$ — the iterate is mostly built

k	t_k	β_k	$\frac{k+2}{2}$	$\frac{k-1}{k+1}$
0	1.000	—	1.000	—
1	1.618	0.000	1.500	0.000
2	2.194	0.282	2.000	0.333
3	2.750	0.434	2.500	0.500
4	3.295	0.531	3.000	0.600
5	3.833	0.599	3.500	0.667
6	4.365	0.649	4.000	0.714
7	4.894	0.688	4.500	0.750
8	5.419	0.719	5.000	0.778
9	5.942	0.744	5.500	0.800
10	6.463	0.765	6.000	0.818
12	7.500	0.798	7.000	0.846
15	9.047	0.833	8.500	0.875
20	11.632	0.876	11.000	0.905
50	26.667	0.948	26.000	0.961
100	51.677	0.974	51.000	0.980

Unified Nesterov: one θ_k recursion handles both regimes

FISTA's t_k recursion ($t_{k+1} = (1 + \sqrt{1 + 4t_k^2})/2$, table on the previous slide) is the *convex-only* version — it gets $O(1/k^2)$ but doesn't speed up when f is strongly convex. Vandenberghe (ECE236C) presents a unified version that uses a different recursion variable θ_k and adapts automatically to whatever strong convexity is present.

Setting. $f = g + h$ with g L -smooth and m -strongly convex (allow $m = 0$), h closed convex with cheap prox.

Update. Initialize $\mathbf{x}^0 = \mathbf{v}^0$, $\theta_0 \in (0, 1]$. For $k = 0, 1, \dots$:

$$\begin{aligned} \theta_k \text{ root of } \frac{\theta_k^2}{t_k} &= (1 - \theta_k)\gamma_k + m\theta_k, \quad \gamma_k = \frac{\theta_{k-1}^2}{t_{k-1}} \\ \mathbf{y} &= \mathbf{x}^k + \frac{\theta_k \gamma_k}{\gamma_k + m\theta_k} (\mathbf{v}^k - \mathbf{x}^k) \\ \mathbf{x}^{k+1} &= \text{prox}_{t_k h}(\mathbf{y} - t_k \nabla g(\mathbf{y})) \\ \mathbf{v}^{k+1} &= \mathbf{x}^k + \frac{1}{\theta_k} (\mathbf{x}^{k+1} - \mathbf{x}^k) \end{aligned}$$

with step size $t_k = 1/L$ (or via line search).

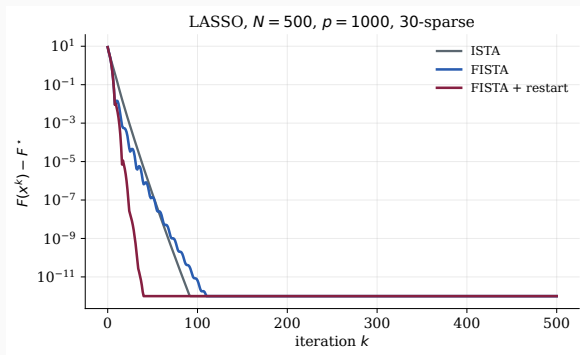
Two regimes from the same algorithm.

- *Convex case* ($m = 0$): $\theta_k \rightarrow 0$ as $1/(k+1)$; the momentum $\beta_k = (1 - \theta_{k-1})/\theta_k$ tracks the standard FISTA form. Rate $O(LD^2/k^2)$.
- *Strongly convex case* ($m > 0$): θ_k converges to $\sqrt{m/L}$ from above; momentum stabilizes at $\beta = (\sqrt{L} - \sqrt{m})/(\sqrt{L} + \sqrt{m})$. Rate $O(LD^2(1 - \sqrt{m/L})^k)$ — the optimal $\sqrt{\kappa}$ bound.

The point. One implementation reaches both rates without any user-side branching. If you don't know m , set $m = 0$ and you still get $O(1/k^2)$. If you can plug in a lower bound for m , you get the linear rate automatically.

Reading. Vandenberghe ECE236C, Lec. 7 ("Accelerated proximal gradient methods"). The classical t_k recursion shown earlier is the $m = 0$ specialization. Tseng (2008) gave a related unified treatment. This update structure is what makes "parameter-free" acceleration work — Lan's framework adds a backtracking rule for L on top.

ISTA vs. FISTA on LASSO — the real plot



A real run (Python script in `figures/make_figures.py`):

- LASSO, $A \in \mathbb{R}^{500 \times 1000}$, 30-sparse $\mathbf{x}^*$, $\lambda = 0.15 \|A^T \mathbf{b}\|_\infty$.
- ISTA, FISTA, FISTA + gradient restart.

Reading the plot.

- ISTA: linear-looking on log scale — the $O(1/k)$ rate looks straight here because we're in the part of the regime where the strongly-convex restricted curvature kicks in.
- FISTA: faster than ISTA, with mild oscillations from the non-monotone momentum.
- FISTA + restart (O'Donoghue &

Roadmap

The First-Order Regime

Gradient Descent: Rates

Heavy-Ball / Polyak Momentum

Nesterov Acceleration

Composite Problems and the Proximal Operator

ISTA and FISTA

Parameter-Free Acceleration

Coding Break: FISTA from scratch

Three Regimes, Three Winners

Frank–Wolfe (Preview)

Wrap-up

The pain point: Nesterov needs to know L

Standard Nesterov uses a fixed step size $\eta = 1/L$. Two practical problems:

- **L is rarely known a priori.** For LASSO it's $\|A\|_2^2$ — one power iteration to estimate, but on large problems even that is costly.
- **The actual local Lipschitz constant is often $\ll L$.** A globally-tight L wastes most iterations on conservative steps.

The fix — backtracking. At each iteration, start with a tentative $L_k \leq L_{k-1}/\rho$ (some $\rho > 1$, say $\rho = 2$), and increase it geometrically until the local quadratic upper bound holds:

$$f(\mathbf{x}^{k+1}) \leq f(\mathbf{y}^k) + \langle \nabla f(\mathbf{y}^k), \mathbf{x}^{k+1} - \mathbf{y}^k \rangle + \frac{L_k}{2} \|\mathbf{x}^{k+1} - \mathbf{y}^k\|^2.$$

This is just FISTA-with-backtracking from Beck & Teboulle (2009). It achieves

$$f(\mathbf{x}^k) - f^* \leq \frac{2L\|\mathbf{x}^0 - \mathbf{x}^*\|^2}{(k+1)^2}$$

without knowing L in advance. Two extensions go further:

- **Nesterov's Universal Gradient Method (2015):** adapts to *Hölder smoothness*, not just Lipschitz.
- **Lan's parameter-free framework:** adapts to both L and the strong-convexity parameter μ , so the same algorithm achieves the optimal rate for both convex and strongly-convex problems.

The next two slides give clean statements of each.

Nesterov's Universal Gradient Method (2015)

Setup. A function f has **Hölder-continuous gradient** of degree $\nu \in [0, 1]$ with constant L_ν if

$$\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\|_2 \leq L_\nu \|\mathbf{x} - \mathbf{y}\|_2^\nu \quad \forall \mathbf{x}, \mathbf{y}.$$

The familiar Lipschitz-gradient case is $\nu = 1$. The case $\nu = 0$ is just a bounded gradient (non-smooth, like $\|\mathbf{x}\|_1$). **Lipschitz smoothness sits at one end of a spectrum.**

Nesterov's universal method. Run accelerated gradient with backtracking: at each iteration find L_k such that

$$f(\mathbf{x}^{k+1}) \leq f(\mathbf{y}^k) + \langle \nabla f(\mathbf{y}^k), \mathbf{x}^{k+1} - \mathbf{y}^k \rangle + \frac{L_k}{2} \|\mathbf{x}^{k+1} - \mathbf{y}^k\|^2 + \frac{\delta}{2}$$

for a target accuracy $\delta > 0$. The user picks δ but not ν or L_ν .

Theorem [Nesterov 2015]. After k iterations, the iterate satisfies

$$f(\mathbf{x}^k) - f^* \leq \frac{c_\nu \cdot L_\nu^{2/(1+3\nu)} \|\mathbf{x}^0 - \mathbf{x}^*\|^{2(1+\nu)/(1+3\nu)}}{k^{(1+3\nu)/2}} + \delta$$

for a constant c_ν that doesn't depend on the problem.

Reading the rate.

- $\nu = 1$ (smooth, classical): exponent $(1+3)/2 = 2$, so $O(1/k^2)$ — Nesterov's classical bound.
- $\nu = 0$ (non-smooth, Lipschitz f): exponent $1/2$, so $O(1/\sqrt{k})$ — subgradient-method rate.
- Intermediate ν : smooth interpolation between the two, automatically.

The point. The algorithm doesn't know ν or L_ν . It *automatically* achieves the optimal rate for whatever smoothness the problem actually has. "Universal" = one method, optimal across the whole Hölder spectrum.

Why this matters. You don't have to classify your problem as "smooth" or "non-smooth" before picking an algorithm. The same accelerated method handles both, and it doesn't lose constants for getting the wrong category.

Lan's parameter-free accelerated method

Setup. Convex L -smooth f . We may or may not have strong convexity $\mu \geq 0$. We want a single algorithm that achieves the optimal rate in *both* regimes without knowing L , μ , or $D = \|\mathbf{x}^0 - \mathbf{x}^*\|$.

The algorithm (paraphrasing Lan's adaptive APG; Lan 2020, Ch. 4 of *First-Order and Stochastic Optimization for ML*). Maintain three sequences $\mathbf{x}^k, \mathbf{z}^k, \mathbf{y}^k$. At each iteration:

$$\begin{aligned} \mathbf{y}^k &= (1 - \alpha_k)\mathbf{x}^k + \alpha_k\mathbf{z}^k \quad (\text{interpolation between primal and dual sequences}) \\ \mathbf{z}^{k+1} &= \arg \min_{\mathbf{z}} \left\{ \alpha_k \langle \nabla f(\mathbf{y}^k), \mathbf{z} \rangle + \frac{\gamma_k}{2} \|\mathbf{z} - \mathbf{z}^k\|^2 \right\} \\ \mathbf{x}^{k+1} &= (1 - \alpha_k)\mathbf{x}^k + \alpha_k\mathbf{z}^{k+1} \end{aligned}$$

with adaptive step parameters α_k, γ_k chosen via a backtracking rule that probes the local Lipschitz behavior of ∇f :

$$\gamma_{k+1} = \rho \cdot \gamma_k \quad (\text{geometric increase}) \text{ until the local quadratic upper bound holds.}$$

Theorem [Lan, simplified statement]. With this adaptive scheme,

$$f(\mathbf{x}^k) - f^* \leq \begin{cases} O\left(\frac{LD^2}{k^2}\right) & (\text{convex case, } \mu = 0) \\ O\left(LD^2 \exp\left(-c\sqrt{\frac{\mu}{L}}k\right)\right) & (\text{strongly-convex case, } \mu > 0) \end{cases}$$

The same algorithm, with no a-priori knowledge of L or μ , achieves the optimal rate in both regimes simultaneously.

Why this is striking. Classical Nesterov requires the user to choose between two algorithms (smooth-convex vs. smooth-strongly-convex), each with its own step sizes. Lan's adaptive scheme picks the right rate for the actual problem at hand — a single “set and forget” implementation.

Reading. Lan (2020), *First-Order and Stochastic Optimization Methods for Machine Learning*, Ch. 3–4. The construction extends to (i) composite problems with proximal operators, (ii) stochastic gradients with noise level σ unknown, and (iii) smoothly-Hölder regimes. The parameter-free principle is the unifying thread of his recent work.

Roadmap

The First-Order Regime

Gradient Descent: Rates

Heavy-Ball / Polyak Momentum

Nesterov Acceleration

Composite Problems and the Proximal Operator

ISTA and FISTA

Parameter-Free Acceleration

Coding Break: FISTA from scratch

Three Regimes, Three Winners

Frank–Wolfe (Preview)

Wrap-up

Coding Break — what to pull up

Notebook: `part2_algorithms/19_proximal_gradient_fista.ipynb`

What we'll build together.

1. **Generate a LASSO instance.** $N = 200, p = 500, k = 20$ true nonzeros, $\sigma = 0.1$ noise.
2. **ISTA from scratch** — 30 lines of code, all standard NumPy.
3. **FISTA from scratch** — add the t -recursion and the look-ahead $\mathbf{y}^k$. Three extra lines of code.
4. **Plot** objective gap vs. iteration for both, on log scale. See the slope-1 vs. slope-2 pattern from the previous slide.
5. **Compare with CVXPY** — same problem, same accuracy. Time both. For $p = 500$, CVXPY wins. Push to $p = 5,000$ and FISTA pulls ahead.
6. **Try TV denoising** — swap the soft-threshold for a TV prox; the rest is identical.

What the code reveals. The non-trivial code is just the prox. The outer loop — gradient, prox, momentum — is exactly 6 lines. This is the compositional power of the framework: implement once, use everywhere.

FISTA in 20 lines

```
import numpy as np

def soft_threshold(v, tau):
    return np.sign(v) * np.maximum(np.abs(v) - tau, 0.0)

def fista_lasso(A, b, lam, n_iter=200):
    n = A.shape[1]
    L = float(np.linalg.norm(A, 2)**2)    # Lipschitz of grad of ||Ax-b||^2/2
    x = np.zeros(n)
    y = x.copy()
    t = 1.0
    losses = []
    for k in range(n_iter):
        x_prev = x.copy()
        grad = A.T @ (A @ y - b)
        x = soft_threshold(y - grad / L, lam / L)    # prox step
        t_next = (1 + np.sqrt(1 + 4 * t**2)) / 2    # momentum coeff
        y = x + ((t - 1) / t_next) * (x - x_prev)  # extrapolate
        t = t_next
        losses.append(0.5 * np.sum((A @ x - b)**2) + lam * np.sum(np.abs(x)))
    return x, losses
```

What's where.

- Lines 1–3: the only “new” math — the prox of ℓ_1 (soft-thresholding).
- Line 9: L -Lipschitz constant via $\|A\|_2^2$. This is the only solver tuning.
- Lines 13–16: the FISTA inner loop: gradient $\rightarrow$ prox $\rightarrow$ momentum.
- Removing line 14–15 and setting $y = x$ in line 16 turns this into ISTA.

Roadmap

The First-Order Regime

Gradient Descent: Rates

Heavy-Ball / Polyak Momentum

Nesterov Acceleration

Composite Problems and the Proximal Operator

ISTA and FISTA

Parameter-Free Acceleration

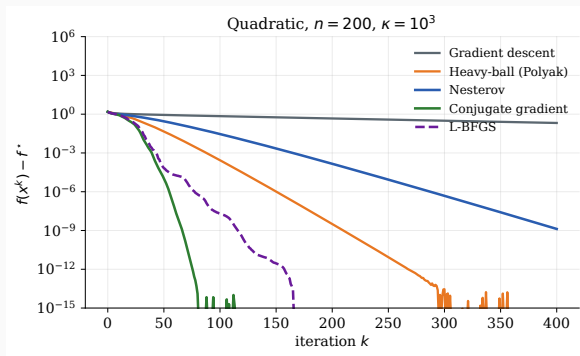
Coding Break: FISTA from scratch

Three Regimes, Three Winners

Frank–Wolfe (Preview)

Wrap-up

Quadratic, $\kappa = 10^3$ — where CG and L-BFGS dominate

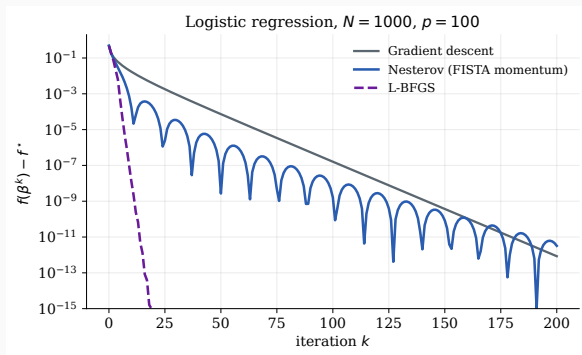


$\min \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x} - \mathbf{b}^\top \mathbf{x}$, $\mathbf{A} \in \mathbb{R}^{200 \times 200}$,
eigenvalues uniformly in $[1, 1000]$.

The pecking order:

- **CG**: hits machine precision in ~ 80 iterations, exactly as the spectrum-adaptive theory predicts.
- **L-BFGS**: nearly as fast as CG. Builds an implicit metric; each step uses only ~ 10 stored gradient differences.
- **Heavy-ball**: linear rate $\sqrt{\kappa}$ with optimal α, β ; hits machine precision around iter 300.
- **Nesterov**: same $\sqrt{\kappa}$ rate, slightly slower constant on this random instance.
- **GD**: only $1 - 1/\kappa$ rate; barely makes

Logistic regression — where L-BFGS pulls ahead

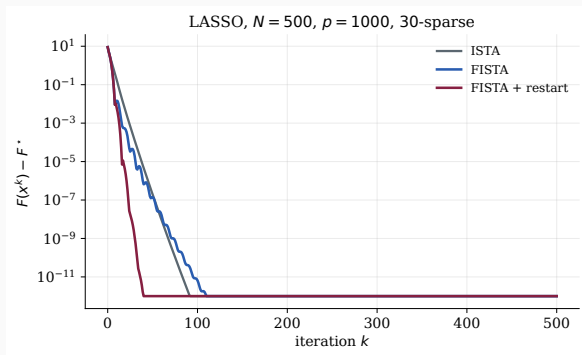


ℓ_2 -regularized logistic regression: $N = 1000$ samples, $p = 100$ features, $\lambda = 0.01$. Smooth (Lipschitz gradient), strongly convex thanks to the ℓ_2 penalty.

Reading the plot.

- **L-BFGS**: reaches machine precision in ~ 25 iterations. Implicit Hessian approximation captures the data-dependent curvature.
- **Nesterov**: visible non-monotone oscillations — exactly the issue we warned about. Still reaches 10^{-12} in ~ 200 iterations.
- **GD**: linear rate, but with κ governed by $A^\top A$ — here $\kappa \sim 100$, so progress is slow.

LASSO — where FISTA + restart wins



LASSO at $N = 500$, $p = 1000$, 30-sparse ground truth. **Composite objective** (smooth + non-smooth) — L-BFGS doesn't apply, CG doesn't apply.

Reading the plot.

- **ISTA**: $O(1/k)$ rate; hits machine precision around iter 100.
- **FISTA**: same per-iteration cost, $O(1/k^2)$ rate; reaches the same accuracy in ~ 100 iterations with mild oscillations.
- **FISTA + gradient restart** (O'Donoghue & Candès 2015): converges in ~ 40 iterations. The restart eliminates oscillation by resetting momentum whenever the inertia stops being a descent

Three regimes summary

Regime	Winner	Why
Smooth quadratic	CG / L-BFGS	Adapt to spectrum; parameter-free
Smooth nonquadratic (e.g., GLMs)	L-BFGS	Implicit Hessian, robust line search
Composite (smooth + non-smooth)	FISTA + restart	Only first-order method that handles prox

When you do reach for Nesterov / FISTA

- Composite problems where L-BFGS can't handle the non-smooth piece.
- Problems too big for Newton-type methods ($n > 10^6$, no Hessian storage).
- Problems with cheap gradient + cheap proximal map but no useful spectral structure.

Practical tip. Implement *multiple* methods (it's ~ 30 lines of Python each), profile against each other on *your* problem at the size you actually care about, then pick. Theoretical rates only tell you which method dominates eventually, not which one is fastest at the accuracy and scale you need today.

Roadmap

The First-Order Regime

Gradient Descent: Rates

Heavy-Ball / Polyak Momentum

Nesterov Acceleration

Composite Problems and the Proximal Operator

ISTA and FISTA

Parameter-Free Acceleration

Coding Break: FISTA from scratch

Three Regimes, Three Winners

Frank–Wolfe (Preview)

Wrap-up

Frank–Wolfe: when projection is hard

Setting. $\min_{\mathbf{x} \in C} f(\mathbf{x})$ with f smooth and C a compact convex set. So far we've assumed projection onto C is cheap. What if it isn't?

Examples where projection is hard.

- $C = \{X : \|X\|_* \leq \tau\}$ (nuclear-norm ball). Projection requires a full SVD — $O(\min(m, n)^2 \max(m, n))$.
- $C = \text{matroid polytope}$, traveling-salesman polytope — exponentially many vertices.
- $C = \{X \succeq 0, \text{tr}(X) = 1\}$ — the projection is a full eigendecomposition.

Frank–Wolfe (1956). Don't project. Instead, at each iteration, solve the *linear* subproblem

$$\mathbf{s}^k := \arg \min_{\mathbf{s} \in C} \langle \nabla f(\mathbf{x}^k), \mathbf{s} \rangle$$

(a **linear oracle** over C), then move toward $\mathbf{s}^k$:

$$\mathbf{x}^{k+1} = (1 - \gamma_k)\mathbf{x}^k + \gamma_k \mathbf{s}^k, \quad \gamma_k = 2/(k+2).$$

Why the trade is favorable.

- Nuclear-ball: linear oracle = top singular vector pair, $O(\text{nnz})$ via power iteration — much cheaper than a full SVD.
- TSP polytope: linear oracle = shortest-path / spanning-tree problem, polynomial.
- Simplex: linear oracle just picks the smallest-cost vertex — $O(n)$.

Rate. $f(\mathbf{x}^k) - f^* \leq 2LD^2/(k+1)$ — $O(1/k)$, slower than FISTA. But for problems where projection is the bottleneck, Frank–Wolfe can be much faster overall. *Bonus:* the iterates are sparse (convex combinations of k vertices).

Final exam Problem 4(iii): Frank–Wolfe for nuclear-norm matrix completion.

Roadmap

The First-Order Regime

Gradient Descent: Rates

Heavy-Ball / Polyak Momentum

Nesterov Acceleration

Composite Problems and the Proximal Operator

ISTA and FISTA

Parameter-Free Acceleration

Coding Break: FISTA from scratch

Three Regimes, Three Winners

Frank–Wolfe (Preview)

Wrap-up

Summary

The hierarchy of first-order rates

On smooth convex problems, the gap $f(\mathbf{x}^k) - f^*$ goes like:

- $O(1/k)$ — gradient descent and ISTA.
- $O(1/k^2)$ — Nesterov / FISTA. The optimal rate (Nemirovski-Yudin lower bound).
- Linear $(1 - 1/\sqrt{\kappa})^k$ — accelerated GD on μ -strongly convex.

For composite $f + g$ with g non-smooth: the same rates carry over via the proximal operator.

The proximal toolkit

- **Prox of $\lambda \|\cdot\|_1$:** soft-thresholding.
- **Prox of $\mathbb{I}_C$:** Euclidean projection onto C .
- **ISTA:** gradient + prox.
- **FISTA:** ISTA + Nesterov momentum.
- **Frank-Wolfe:** when projection is hard but the linear oracle is cheap.

Further reading

- **Nesterov** (1983), “A method for solving the convex programming problem with convergence rate $O(1/k^2)$,” *Soviet Math. Dokl.* — the original.
- **Beck & Teboulle** (2009), “A fast iterative shrinkage-thresholding algorithm for linear inverse problems,” *SIAM J. Imaging Sci.* 2(1), 183–202. — FISTA.
- **Nesterov** (2015), “Universal gradient methods for convex optimization problems,” *Math. Programming* 152(1), 381–404. — Universal method for Hölder gradients.
- **Lan** (2020), *First-Order and Stochastic Optimization Methods for Machine Learning*, Springer. — Ch. 3–4 cover parameter-free / adaptive accelerated methods.
- **Parikh & Boyd** (2014), *Proximal Algorithms*, Foundations and Trends in Optimization 1(3), 127–239. — the proximal-operator encyclopedia.
- **Bubeck** (2015), *Convex Optimization: Algorithms and Complexity*, Foundations and Trends in Machine Learning 8(3-4), 231–357. — modern unified treatment of first-order methods.
- **Jaggi** (2013), “Revisiting Frank–Wolfe: projection-free sparse convex optimization,” ICML.
- **O’Donoghue & Candès** (2015), “Adaptive restart for accelerated gradient schemes,” *Found. Comput. Math.* 15, 715–732. — restart strategies for FISTA.
- **Boyd & Vandenberghe** (2004), *Convex Optimization*, Cambridge UP, Ch. 9 — gradient method